

Scaled Relative Graph Toolbox

MATLAB Reference Documentation

Frequency-domain analysis of SISO and MIMO LTI systems
using Scaled Relative Graphs, Beltrami-Klein geometry,
and hyperbolic convex-hull methods.

Eder Baron-Prada, Adolfo Anta, Thomas Chaffey, Alberto Padoan.

Version 1.0

0 Contents

1	Introduction	3
1.1	Toolbox Structure	3
1.2	Quick Start	4
2	Function Index	4
3	Mathematical Background	5
3.1	Gain, Phase, and the Scaled Relative Graph	5
3.2	Beltrami-Klein (BK) Mapping	6
3.3	Inverse BK Mapping	6
3.4	Soft SRG, Hard SRG, and Their Relationship	6
3.5	Feedback Stability Criterion	7
4	Core Computation Functions	8
4.1	srg_compute	8
4.2	srg_scaled_graph	10
4.3	srg_hard	11
4.4	srg_homotopy	12
4.5	srg_stability_margin	13
4.6	srg_bounding_circle	13
4.7	srg_qsr_multiplier	14
5	Visualization Functions	15
5.1	srg_plot_gauss	15
5.2	srg_plot_beltrami	15
5.3	srg_plot_compare	16
5.4	srg_plot_3d	16
5.5	srg_plot_3d_surface	16
5.6	srg_plot_3d_compare	17
5.7	srg_plot_3d_beltrami	17
5.8	srg_plot_3d_beltrami_compare	17
5.9	srg_plot_witness_3d	18
6	Style System and Figure Export	18
6.1	srg_style	18
6.2	srg_apply_style	19
7	Internal Helper Functions (bg/)	19
7.1	beltrami_map_matrix	19
7.2	beltrami_map_scalar	19
7.3	beltrami_inv	20
7.4	srg_refine_chords	20
7.5	srg_export	21
7.6	field_of_values	21
7.7	srg_to_polyshape	22
8	Example Scripts	22
8.1	Example 1 — Frequency-wise SRG (example_01_freqwise_srg.m)	22
8.2	Example 2 — Soft SRG (example_02_soft_srg.m)	22
8.3	Example 3 — Hard SRG (example_03_hard_srg.m)	22
8.4	Example 4 — QSR Multiplier and Export (example_04_qsr_export.m)	23

9	Conventions and Data Format	23
9.1	Cell Array Convention	23
9.2	Frequency Convention	24
9.3	Feedback Partner Convention	24
9.4	QSR Multiplier Format	24
9.5	Output Table Fields (<code>rawdata</code>)	24
10	Dependencies and Licenses	24
10.1	Third-Party Code	24
10.2	MATLAB Requirements	25

1 Introduction

The **Scaled Relative Graph (SRG) Toolbox** is a MATLAB toolbox for computing and visualizing the Scaled Relative Graph [7, 11] of linear time-invariant (LTI) systems. It supports both SISO and MIMO systems and provides tools for:

- Frequency-wise SRG computation in the Beltrami-Klein disk and the Gauss (complex) plane.
- Soft SRG via the hyperbolic convex hull of frequency-wise slices.
- Hard SRG for systems with unstable poles or non-minimum-phase zeros.
- Stability margin analysis and SRG-based feedback stability assessment.
- Homotopy paths of scaled SRGs.
- Frequency-wise circular (Q, S, R) multiplier extraction (solver-free).
- 2D and 3D visualization with a unified, theme-aware style system.
- Publication-ready figure export (PDF/EPS/PNG/TikZ).

1.1 Toolbox Structure

The public API lives at the toolbox root; lower-level helpers and the plotting/style infrastructure live in `bg/`, and the runnable example scripts live in `examples/`. Run `setup.m` once to add all three directories (root, `bg/`, `examples/`) to the MATLAB path.

```
tb/
|-- Contents.m           % Function index (help Contents)
|-- setup.m             % Add root + bg/ + examples/ to the path
|-- srg_compute.m       % Core frequency-wise SRG computation
|-- srg_hard.m          % Hard SRG boundary
|-- srg_homotopy.m      % Homotopy of tau*G
|-- srg_scaled_graph.m  % Soft SRG (hyperbolic convex hull)
|-- srg_bounding_circle.m % Minimum enclosing circle of an SRG set
|-- srg_qsr_multiplier.m % Frequency-wise (Q,S,R) multiplier fit
|-- srg_export.m        % Publication export (PDF/EPS/PNG/TikZ)
|-- srg_plot_gauss.m    % 2D Gauss-plane plot
|-- srg_plot_beltrami.m % 2D Beltrami-Klein plot
|-- srg_plot_compare.m  % Multi-SRG overlay
|-- srg_plot_3d.m       % 3D wire/filled plot
|-- srg_plot_3d_surface.m % 3D surface plot
|-- srg_plot_3d_compare.m % 3D comparison with intersection
|-- srg_plot_3d_beltrami.m % 3D BK-disk plot
|-- srg_plot_3d_beltrami_compare.m % 3D BK-disk comparison
|-- srg_plot_witness_3d.m % Min-distance witness segments (3D)
|-- srg_stability_margin.m % Frequency-wise stability margin
|-- examples/           % Runnable demonstration scripts
|   |-- example_01_freqwise_srg.m % Example: frequency-wise SRG
|   |-- example_02_soft_srg.m    % Example: soft SRG
|   |-- example_03_hard_srg.m    % Example: hard SRG
|   |-- example_04_qsr_export.m  % Example: QSR multiplier + export
|-- bg/                 % Internal helpers + plot/style backend
|   |-- beltrami_inv.m
|   |-- beltrami_map_matrix.m
|   |-- beltrami_map_scalar.m
|   |-- field_of_values.m
|   |-- srg_refine_chords.m
```

```
|-- srg_to_polyshape.m
|-- srg_style.m
'-- srg_apply_style.m
```

1.2 Quick Start

```
1 [caption={Minimal example: SISO SRG}]
2 G = tf([1 2], [1 3 2]);
3 [W, A, gplus, gminus, rawdata] = srg_compute(G, -3, 3, 200, 1);
4
5 figure;
6 srg_plot_gauss(G, gplus, gminus, 1, 200);
7 title('SRG of G');
```

```
1 [caption={Minimal example: MIMO feedback pair}]
2 G1 = [tf([1 2],[1 3 2]) tf([0.1],[1 1]);
3       tf([0.05],[1 2])  tf([1],[1 1])];
4 G2 = [tf([1 1],[1 5])   tf([1],[3 2 2]);
5       tf([1],[1 2])    -tf([0.5],[1 2])];
6
7 [~, ~, gp1, ~, rd1] = srg_compute(G1, -3, 3, 100, 500);
8 [~, ~, gp2, ~, rd2] = srg_compute(-inv(G2), -3, 3, 100, 500);
9
10 figure;
11 srg_plot_3d_compare(rd1, gp1, rd2, gp2, 0, 0, ...
12     'Color1', 1, 'Color2', 5, 'FaceAlpha', 0.6);
```

2 Function Index

Function	Purpose
srg_compute	Core frequency-wise SRG computation
srg_scaled_graph	Soft SRG (hyperbolic convex hull)
srg_hard	Hard SRG for unstable/NMP systems
srg_homotopy	SRG of τG for multiple τ
srg_stability_margin	Frequency-resolved stability margin + witness pair
srg_bounding_circle	Minimum enclosing circle of an SRG set
srg_qsr_multiplier	Frequency-wise (Q, S, R) multiplier fit
srg_export	Publication export (PDF/EPS/PNG/TikZ)
srg_plot_gauss	2D SRG in Gauss plane
srg_plot_beltrami	2D SRG in Beltrami-Klein disk
srg_plot_compare	Multi-SRG overlay
srg_plot_3d	3D wire/filled plot
srg_plot_3d_surface	3D smooth surface
srg_plot_3d_compare	3D two-SRG comparison
srg_plot_3d_beltrami	3D BK-disk plot
srg_plot_3d_beltrami_compare	3D BK-disk comparison
srg_plot_witness_3d	3D min-distance witness segments
srg_style	Style configuration struct
srg_apply_style	Apply style to current axes

Function	Purpose
<code>srg_to_polyshape</code>	Complex boundary to polyshape
<code>beltrami_map_matrix</code>	BK mapping (matrix operator)
<code>beltrami_map_scalar</code>	BK mapping (scalar eigenvalues)
<code>beltrami_inv</code>	Inverse BK mapping ($g^\pm$)
<code>field_of_values</code>	Numerical range of a matrix
<code>srg_refine_chords</code>	Chord subdivision before the inverse BK map

3 Mathematical Background

3.1 Gain, Phase, and the Scaled Relative Graph

For signals $u, y \in L_2^n$ with $u \neq 0$, define the **gain** and **phase** of the input-output pair as

$$\rho(u, y) := \frac{\|y\|}{\|u\|}, \quad \theta(u, y) := \arccos\left(\frac{\langle u, y \rangle}{\|u\| \|y\|}\right), \quad (1)$$

with $\theta(u, y) = 0$ when $y = 0$. Each pair (ρ, θ) encodes a point $\rho e^{\pm j\theta} \in \mathbb{C}$: the modulus captures amplitude change and the argument captures the angular misalignment between input and output.

For a linear operator H , the **Scaled Relative Graph** (SRG) is the set of all such complex numbers over all non-zero inputs [10, 11]:

$$\text{SRG}(H) := \left\{ \frac{\|Hu\|}{\|u\|} e^{\pm j\theta(u, Hu)} \mid u \in L_2^n, u \neq 0 \right\}. \quad (2)$$

The SRG is always symmetric with respect to the real axis. Its modulus encodes gain variation across input directions and its argument encodes phase variation; together they generalize the Nyquist diagram to MIMO systems [7].

Frequency-wise decomposition for LTI systems

The Fourier transform diagonalizes convolution, so the input-output map of an LTI system $H(s) \in \mathcal{RH}_\infty^{n \times n}$ decouples across frequencies. The SRG at a single frequency $\omega \in \mathbb{R}$ is:

$$\text{SRG}(H(j\omega)) := \left\{ \frac{\|H(j\omega)u\|}{\|u\|} e^{\pm j \arccos\left(\frac{\text{Re}\{\langle H(j\omega)u, u \rangle\}}{\|H(j\omega)u\| \|u\|}\right)} \mid u \in \mathbb{C}^n, u \neq 0 \right\} \subset \mathbb{C}. \quad (3)$$

The geometric complexity of (3) grows with system dimension n : for SISO systems ($n = 1$), each frequency slice is a conjugate pair of points; for $n = 2$, a pair of conjugate curves; for $n \geq 3$, a pair of conjugate filled regions.

The full operator SRG is the hyperbolic convex hull of all frequency-wise slices:

$$\text{SRG}(H) = \text{co}_h\left(\bigcup_{\omega \in \mathbb{R}} \text{SRG}(H(j\omega))\right)$$

, where co_h denotes the hyperbolic convex hull in the Beltrami-Klein model.

Maximum gain and phase

Two scalar quantities summarize the SRG geometry at each frequency and underpin the small-gain and small-phase stability conditions:

$$\sigma_{\max}(H(j\omega)) := \max_{\|u\|=1} \|H(j\omega)u\|, \quad (4)$$

$$\alpha_{\max}(H(j\omega)) := \max_{\|H(j\omega)u\| \neq 0, \|u\|=1} \arccos\left(\frac{\operatorname{Re}\{\langle H(j\omega)u, u \rangle\}}{\|H(j\omega)u\| \|u\|}\right). \quad (5)$$

$\sigma_{\max}$ is the largest singular value and governs the small-gain theorem. $\alpha_{\max}$ is the maximum input-output angle and governs the small-phase theorem [3]; it is well-defined for every operator regardless of whether the numerical range contains the origin (unlike the classical sectorial phase).

3.2 Beltrami-Klein (BK) Mapping

The toolbox maps the frequency-response matrix $G(j\omega)$ into the unit disk $\mathbb{D} = \{z \in \mathbb{C} : |z| \leq 1\}$ via the **Beltrami-Klein mapping**. For a matrix $T \in \mathbb{C}^{n \times n}$:

$$\Phi(T) = (I + T^*T)^{-1/2} (T^* - iI) (T - iI) (I + T^*T)^{-1/2}, \quad (6)$$

and for a scalar $\lambda \in \mathbb{C}$ (e.g., an eigenvalue):

$$f(\lambda) = \frac{(\bar{\lambda} - i)(\lambda - i)}{1 + \bar{\lambda}\lambda}. \quad (7)$$

The key geometric property of the BK model is that **Euclidean convex hulls in $\mathbb{D}$ coincide with hyperbolic convex hulls**, because hyperbolic geodesics in the BK model are Euclidean straight-line segments. This means that the soft SRG can be computed simply as the Euclidean convex hull of all frequency-wise BK-disk images.

3.3 Inverse BK Mapping

The inverse map g^{-1} recovers SRG points in the Gauss plane from BK disk coordinates via two branches:

$$g^{\pm}(z) = \frac{\operatorname{Im}(z) \pm i\sqrt{1 - |z|^2}}{\operatorname{Re}(z) - 1}, \quad (8)$$

where g^+ gives the upper half-plane boundary and g^- the lower (conjugate) boundary of the SRG. The toolbox calls this map via `beltrami_inv`.

Because $g^{\pm}$ is strongly nonlinear, a straight chord in the BK plane does not, in general, map to a straight chord (or even a gently curved segment) in the Gauss plane. `field_of_values` returns only polygon vertices for normal matrices (the field of values of a normal matrix is exactly the convex hull of its eigenvalues), so BK-plane chords between vertices can be long. `srg_refine_chords` subdivides such chords, within the (linear) BK disk, before `beltrami_inv` is applied; see Section 7.4.

3.4 Soft SRG, Hard SRG, and Their Relationship

Three SRG variants appear in the toolbox, reflecting different classes of input signals and operator assumptions.

Frequency-wise SRG. The set (3) at a fixed frequency ω . Computed by `srg_compute` for each point in the frequency grid; stored in the `gplus/gminus` cell arrays.

Soft SRG [10]. Defined over L_2 signals and the full time horizon $[0, \infty)$; requires $u \in L_2^n$ and $y = Hu \in L_2^n$. For an LTI system in $\mathcal{RH}_\infty$, the soft SRG equals the hyperbolic convex hull of all frequency-wise slices:

$$\text{SG}(H) := \text{co}_h \left(\bigcup_{\omega \in \mathbb{R}} \text{SRG}(H(j\omega)) \right).$$

Computed by `srg_scaled_graph`.

Hard SRG [9]. Defined over extended L_{2e} signals using truncated gain and phase quantities ρ_T, θ_T over finite horizons $T > 0$. The hard SRG always satisfies $\text{SG}(H) \subseteq \overline{\text{SG}_e(H)}$. It accounts for:

- **Unstable poles:** max gain $\rightarrow \infty$, capped at a finite value `InfVal`.
- **Non-minimum-phase zeros:** min gain = 0.

Computed by `srg_hard`.

3.5 Feedback Stability Criterion

Consider the negative feedback interconnection of H_1 and H_2 shown in Figure 1.

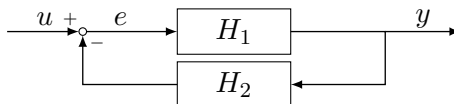


Figure 1: Negative feedback interconnection of H_1 and H_2 .

The SRG stability theorem certifies L_2 -stability via geometric separation of SRGs [2, 6]. Extensions to decentralized loops [1] and to power-electronics-dominated grids [4, 5] follow the same separation principle.

Frequency-wise GFT for LTI systems

For $H_1(s), H_2(s) \in \mathcal{RH}_\infty^{n \times n}$, the closed loop is L_2 -stable if, at every frequency $\omega \in [0, \infty)$ and for all $\tau \in (0, 1]$:

$$\text{SRG}(H_1(j\omega))^{-1} \cap (-\tau \text{SRG}(H_2(j\omega))) = \emptyset, \quad (9)$$

where $\text{SRG}(H)^{-1} := \{z^{-1} \mid z \in \text{SRG}(H), z \neq 0\}$. The homotopy parameter τ continuously deforms one SRG toward the origin; strict separation over the full range $\tau \in (0, 1]$ certifies stability without requiring the full Nyquist encirclement count.

Toolbox convention

In practice, `srg_compute` is called on H_1^{-1} and $-H_2$ separately. The SRGs must be disjoint for the feedback pair to be stable. The stability margin (minimum distance between boundaries at each frequency) is computed by `srg_stability_margin`, and the closest-pair endpoints it returns can be drawn as witness segments by `srg_plot_witness_3d`.

Hard SRG stability

For operators that need not be L_2 -stable, the hard separation condition

$$\text{dist}(\text{SG}_e^{-1}(H_1), -\text{SG}_e(H_2)) > 0$$

is sufficient [9] and is evaluated geometrically from the `srg_hard` output via fill plots or `srg_plot_compare`.

4 Core Computation Functions

4.1 srg_compute

Purpose

Computes the frequency-wise SRG of an LTI system or constant matrix. This is the primary entry point for all SRG analyses.

Signature

```

1 [W, A, gplus, gminus, rawdata] = srg_compute(TransferFcn)
2 [W, A, gplus, gminus, rawdata] = srg_compute(TransferFcn, ...
3     rangemin, rangemax, estpoints, points)
4 [...] = srg_compute(..., 'FreqScale', 'log' | 'linear' | 'auto')
5 [...] = srg_compute(..., 'Inverse', true|false)
6 [...] = srg_compute(..., 'Refine', true|false, 'RefineTol', 0.05)

```

Inputs

Argument	Type	Description
TransferFcn	tf/ss/double	Transfer function, state-space model, or constant matrix.
rangemin	double	Log base-10 of the minimum frequency [Hz] (or minimum linear frequency if <code>FreqScale='linear'</code>). Optional; defaults to automatic selection.
rangemax	double	Log base-10 of the maximum frequency [Hz] (or maximum linear frequency). Optional; defaults to automatic selection.
estpoints	int	Number of frequency evaluation points (default: 200). Ignored when <code>FreqScale='auto'</code> , since the frequency vector is then determined by the system dynamics.
points	int	Angular resolution for the field of values (default: 64). Set 1 for SISO (each slice is a single point). Use 32–2000 for MIMO depending on required smoothness. Also used as the chord-refinement density when <code>Refine=true</code> (see below).

Name-Value Arguments

Option	Default	Description
FreqScale	'auto'	Frequency spacing: 'log' (logspace between <code>rangemin</code> and <code>rangemax</code>), 'linear' (linspace), or 'auto'. In 'auto' mode the frequency vector is obtained from MATLAB's <code>nyquist</code> automatic selector.
Inverse	false	If <code>true</code> , returns the SRG of the <i>inverse</i> operator, $\text{SRG}(T^{-1})$.

Option	Default	Description
Refine	true	If true , subdivides long Beltrami-Klein-plane chords via srg_refine_chords before every call to beltrami_inv (see the note below and Section 7.4). Set to false to skip this step, e.g. for speed when the operator is known to be non-normal at every frequency in the sweep.
RefineTol	0.05	Maximum real- or imaginary-part step allowed between consecutive BK-plane boundary points before srg_refine_chords subdivides that chord. Only used when Refine=true ; smaller values give a closer approximation of curved (non-normal) boundary segments at the cost of more inserted points.

Outputs

Output	Type	Description
W	cell	$1 \times N$ cell array; W{i} contains the BK-disk numerical range boundary at frequency i , after chord refinement (if Refine=true). When Inverse=true , contains the BK-disk numerical range of $\text{inv}(T(j\omega))$ instead of the original operator's.
A	cell	$1 \times N$ cell array of eigenvalues in the BK disk. When Inverse=true , contains the eigenvalues of the mapped $\text{inv}(T(j\omega))$ rather than those of the original operator.
gplus	cell	$1 \times N$ cell array; upper SRG boundary in the Gauss plane. When Inverse=true , this is the upper boundary of $\text{SRG}(T^{-1})$, computed directly from $\text{inv}(T(j\omega))$.
gminus	cell	$1 \times N$ cell array; lower SRG boundary in the Gauss plane. When Inverse=true , this is the lower boundary of $\text{SRG}(T^{-1})$.
rawdata	table	Table with columns freq , maxphase , minphase , norminf , norminfm . When Inverse=true , an additional logical column zero_interior is included, flagging the frequency indices at which the computed inverse boundary contains non-finite points (i.e. where $\text{SRG}(T^{-1})$ is unbounded; see the note below).

Constant matrices. When **TransferFcn** is a **double** matrix, the SRG is computed once and replicated across all frequency points. **gplus{i}** is identical for all i ; the frequency axis in **rawdata** uses **logspace(-2,3,200)** by default and carries no physical significance.

Unbounded inverse SRGs. When `Inverse=true`, `srg_compute` checks at every frequency slice whether the resulting boundary (`gplus/gminus`) contains any non-finite points. This happens when the Beltrami–Klein image of $\text{inv}(T(j\omega))$ touches the singular point of the inverse BK map — which occurs precisely when $\text{SRG}(T)$ comes close to containing the origin. In that case $\text{SRG}(T^{-1})$ is (partially) unbounded and contains the point at infinity. A `warning('srg_compute:UnboundedInverseSRG', ...)` is then issued, reporting the frequency range(s) over which this occurs; the same information is recorded in `rawdata.zero_interior`.

Example

```

1  G = tf([1], [1 2 1]);
2  % Automatic frequency range
3  [W, A, gp, gm, rd] = srg_compute(G);
4  % Manual range
5  [W, A, gp, gm, rd] = srg_compute(G, -2, 3, 300, 1);
6  figure;
7  srg_plot_gauss(G, gp, gm, 1, length(rd.freq));
8  srg_plot_beltrami(G, W, A, 2, length(rd.freq));
9
10 % Inverse SRG, SRG(G^-1)
11 [Wi, A, gpi, gmi, rdi] = srg_compute(G, -2, 3, 300, 1, 'Inverse',
    true);
12 if any(rdi.zero_interior)
13     disp('Inverse SRG is unbounded (contains infinity) at some
        frequencies.')
14 end
15 % Wi now holds the BK-disk boundary of T^-1, not of T
16
17 % Normal operator with real eigenvalues -- chord refinement (default:
18 % on) is what keeps the mapped SRG from collapsing into a degenerate
19 % curve. Shown explicitly here with a tighter tolerance.
20 H = diag([3, -2/3, 1]);
21 [W, A, gp, gm, rd] = srg_compute(H, [], [], 1, 32, 'RefineTol', 0.02)
    ;

```

4.2 srg_scaled_graph

Purpose

Computes the **soft SRG** [10] (also called the Soft Scaled Graph) as the hyperbolic convex hull of all frequency-wise BK-disk slices.

Signature

```

1  [sg_plus, sg_minus, hull_bk] = srg_scaled_graph(W)
2  [sg_plus, sg_minus, hull_bk] = srg_scaled_graph([], ...
3      'InputDomain', 'Gauss', 'GPlus', gplus, 'GMinus', gminus)

```

Inputs

Argument	Type	Description
W	cell	Cell array of BK-disk boundary curves from <code>srg_compute</code> . Pass [] when using Gauss-plane input.
InputDomain	string	'BK' (default) or 'Gauss'.
GPlus	cell	Required when InputDomain='Gauss'.
GMinus	cell	Required when InputDomain='Gauss'.
RefinementPoints	int	Points per hull edge for the BK-to-Gauss mapping (default: 100).

Outputs

Output	Type	Description
sg_plus	cell	{1} cell containing the upper boundary of the soft SRG in the Gauss plane.
sg_minus	cell	{1} cell containing the lower boundary.
hull_bk	complex	Complex vector of the convex hull boundary in the BK disk.

Algorithm. (1) Pool all BK-disk points across frequencies. (2) Compute the Euclidean convex hull in the BK disk (= hyperbolic convex hull by the BK model property). (3) Interpolate hull edges (nonlinear inverse map requires dense sampling). (4) Apply g^{-1} to map to the Gauss plane.

4.3 srg_hard

Purpose

Computes the **hard SRG** [9] boundary of an LTI system, accounting for unstable poles and non-minimum-phase zeros.

Signature

```

1 [srg, srg_upper] = srg_hard(G, alphas, phis, frequencies)
2 [srg, srg_upper] = srg_hard(G, alphas, phis, frequencies, 'InfVal',
   50)
```

Inputs

Argument	Type	Description
G	tf/ss	Transfer function or state-space model.
alphas	double	Row vector of real base points for the alpha sweep. Typical: <code>linspace(-10, 10, 100)</code> .
phis	double	Row vector of angles in $[0, \pi]$ for boundary resolution. Typical: <code>linspace(0, pi, 200)</code> .
frequencies	double	Frequency grid in rad/s. Typical: <code>logspace(-3, 3, 500)</code> .

Argument	Type	Description
InfVal	double	Finite replacement for infinite gain (default: 10).

Outputs

Output	Type	Description
srg	complex	Full SRG boundary (upper + mirrored lower half).
srg_upper	complex	Upper half-plane boundary only.

Algorithm

1. For each α_i in **alphas**: construct $G(s) - \alpha_i I$ and compute $\sigma_{\min}$, $\sigma_{\max}$ via SVD sweep. If $G - \alpha_i I$ is unstable, set $\sigma_{\max} = \text{InfVal}$. If $G - \alpha_i I$ has non-minimum-phase zeros, set $\sigma_{\min} = 0$.
2. Find the real interval $[a, b]$ contained in all max-gain circles $[\alpha_i - r_i^{\max}, \alpha_i + r_i^{\max}]$.
3. For each angle ϕ in **phis**: intersect all max-gain circles centered at α_i to find the tightest outer radius from $z_0 = (a + b)/2$.
4. Carve out min-gain circles from the resulting contour.
5. Mirror the upper boundary to obtain the full boundary.

Example

```

1 s = tf('s');
2 G = (s+7)/(s-1); % unstable pole
3
4 alphas = linspace(-10, 10, 100);
5 phis = linspace(0, pi, 200);
6 freqs = logspace(-3, 3, 500);
7
8 [srg_bnd, ~] = srg_hard(G, alphas, phis, freqs, 'InfVal', 20);
9
10 figure;
11 fill(real(srg_bnd), imag(srg_bnd), [0.85 0.32 0.10], ...
12      'FaceAlpha', 0.4, 'EdgeColor', 'none');
13 axis equal; grid minor;
14 xlabel('Re'); ylabel('Im');
15 title('Hard SRG');
```

4.4 srg_homotopy

Purpose

Computes frequency-wise SRGs of the scaled system $G_\tau = \tau \cdot G$ for multiple τ values. Useful for visualizing how the SRG shrinks toward the origin as $\tau \rightarrow 0$.

Signature

```

1 homotopy_data = srg_homotopy(G, tau_values, rangemin, rangemax, ...
2                             estpoints, points)
```

Output

A struct array with one element per τ value. Each element has fields: `tau`, `W`, `A`, `gplus`, `gminus`, `rawdata`.

Example

```

1 G = tf([1 2], [1 3 2]);
2 hd = srg_homotopy(G, [1.0, 0.6, 0.3], -2, 3, 200, 64);
3
4 figure;
5 for k = 1:numel(hd)
6     srg_plot_3d_surface(hd(k).rawdata, hd(k).gplus, 0, 0, ...
7                         'Color', k, 'FaceAlpha', 0.6);
8     hold on;
9 end

```

4.5 srg_stability_margin*Purpose*

Computes the frequency-resolved stability margin: the minimum Euclidean distance between two SRG boundaries at each frequency, together with the closest-pair endpoints on each boundary (the witness pair). Pairwise distances are computed without the Statistics Toolbox via a `bsxfun`-based expansion.

Signature

```

1 [stab_margin, freq, x1, x2] = srg_stability_margin(gplus1, gplus2,
    rawdata)

```

Inputs / Outputs

Name	Type	Description
<code>gplus1/2</code>	cell	SRG boundaries from <code>srg_compute</code> . Accepts scalar cells (constant operator) or frequency-indexed cells.
<code>rawdata</code>	table/vector	Frequency vector or table from <code>srg_compute</code> .
<code>stab_margin</code>	double	Minimum distance per frequency.
<code>freq</code>	double	Frequency vector.
<code>x1, x2</code>	complex	Closest points on each SRG at every frequency (witness pair).

4.6 srg_bounding_circle*Purpose*

Computes the minimum enclosing circle of a set of complex SRG boundary points. Because SRGs are symmetric about the real axis, the center is constrained to the real line; the problem reduces to a 1D minimax solved with `fminbnd` (base MATLAB, no toolboxes).

Signature

```

1 [center, radius] = srg_bounding_circle(srg)
2 [center, radius] = srg_bounding_circle(srg, 'Plot', true)

```

Name	Type	Description
<code>srg</code>	complex	Column vector of SRG boundary points.
<code>Plot</code>	logical	Overlay the circle on current axes (default: false).
<code>center</code>	double	Real-valued circle center.
<code>radius</code>	double	Minimum enclosing radius.

4.7 srg_qsr_multiplier*Purpose*

Extracts a frequency-wise circular (Q, S, R) multiplier from the SRG. At each frequency the tightest circular region $\Pi(c, r)$ enclosing the SRG slice is found via `srg_bounding_circle`; rational transfer functions are then fit to the parameter sequences $c(\omega)$ and $R(\omega) = r^2 - c^2$. Fitting uses Sanathanan-Koerner (SK) weighted least squares with AIC order selection.

For the interior disk $\mathbb{D}^{\text{int}}(c, r)$, the circular multiplier (Proposition 1 of [6]) is

$$\Pi_{\text{int}}(c, r) = \begin{bmatrix} -1 & c \\ c & r^2 - c^2 \end{bmatrix}, \quad Q = -1, S = c, R = r^2 - c^2, \quad (10)$$

and Π_{int} is positive-negative (soft-hard equivalent) iff $r > |c|$.

Signature

```

1 out = srg_qsr_multiplier(gplus, rawdata)
2 out = srg_qsr_multiplier(gplus, rawdata, 'MaxOrder', 8, ...
3      'SKIter', 20, 'Plot', true)

```

Name-Value Arguments

Option	Default	Description
<code>MaxOrder</code>	8	Maximum TF order searched (AIC selects within 1...MaxOrder).
<code>SKIter</code>	20	SK iterations per candidate order.
<code>Plot</code>	false	Diagnostic plot of c , R data + fits, and $r > c $.

Output struct fields

Field	Type	Description
<code>freq</code>	double	Frequency vector [Hz].
<code>c</code>	double	Per-frequency circle center.
<code>r</code>	double	Per-frequency circle radius.
<code>R</code>	double	Per-frequency $R = r^2 - c^2$.
<code>pos_neg</code>	logical	True where $r > c $ (multiplier is positive-negative).

Field	Type	Description
S_tf	tf	Fitted TF for $c(\omega)$ (S parameter).
R_tf	tf	Fitted TF for $R(\omega)$ (R parameter), signed-absolute overbounded.
S_ord, R_ord	int	Selected TF orders.
R_scale	double	Multiplicative overbound factor $k \geq 1$ applied to R_tf .
Pi	struct	Multiplier struct with Q=-1, S=S_tf, R=R_tf.

Example

```

1 G = tf([1], [1 2 1]);
2 [~,~,gp,~,rd] = srg_compute(G, -2, 3, 200, 32);
3 out = srg_qsr_multiplier(gp, rd, 'Plot', true);
4 fprintf('Selected orders: S=%d R=%d\n', out.S_ord, out.R_ord);

```

5 Visualization Functions

All plotting functions share a common set of behaviors:

- **SISO detection** is data-driven: if every cell in `gplus` contains a single unique complex value, the system is treated as SISO regardless of the `TransferFcn` type.
- **Constant matrices** (double inputs to `srg_compute`) produce a static SRG plotted once.
- **Theming** is controlled via the `'Theme'` option (see Section 6).
- All functions call `hold` on internally so they can be layered in a single figure.

5.1 srg_plot_gauss

Plots the SRG in the Gauss (complex) plane.

```

1 srg_plot_gauss(TransferFcn, gplus, gminus, color, estpoints)
2 srg_plot_gauss(..., 'Fill', true, 'FaceAlpha', 0.5, 'Theme', 'dark')

```

Option	Default	Description
color	—	Color index (1–8) or RGB triplet.
Fill	false	Fill the SRG interior. For SISO: fills the region swept by the <code>gplus/gminus</code> traces. For MIMO: fills each frequency slice.
FaceAlpha	0.5	Fill transparency.
Theme	'default'	Style theme.

5.2 srg_plot_beltrami

Plots the SRG in the Beltrami-Klein disk.

```

1 srg_plot_beltrami(TransferFcn, W, A, color, estpoints)
2 srg_plot_beltrami(..., 'Fill', true, 'FaceAlpha', 0.4)

```

For SISO systems, draws the BK trajectory of the scalar across frequency as a continuous curve inside the disk with frequency markers.

5.3 srg_plot_compare

Overlays multiple SRGs with automatic color cycling and a legend. Accepts either:

- **Cell arrays** (gplus from srg_compute): plots per-frequency curves or SISO trajectories.
- **Complex vectors** (srg from srg_hard): plots as filled polyshape regions.

```
1 srg_plot_compare(srg1, srg2, ...)
2 srg_plot_compare(srg1, srg2, 'Names', ["Plant","Controller"], ...
3   'Fill', true, 'FaceAlpha', 0.6, 'Theme', 'publication')
```

Option	Default	Description
Names	auto	String array of legend entries.
Theme	'default'	Style theme.
Mode	'filled'	'filled' or 'boundary' (for vector inputs).
FaceAlpha	from theme	Fill transparency.
Fill	false	Fill interior of cell-array inputs (MIMO slices or SISO region).
Title	"	Plot title.

5.4 srg_plot_3d

3D wire or filled plot of SRG boundaries stacked along the frequency axis (log-scaled z -axis).

```
1 srg_plot_3d(rawdata, gplus, fmin, fmax, color)
2 srg_plot_3d(..., 'Fill', true, 'FaceAlpha', 0.4, 'Mirror', true, ...
3   'FillTightness', 0, 'ShowBoundary', true)
```

Option	Default	Description
fmin/fmax	0	Frequency window (0 = full range).
Mirror	true	Plot the conjugate (lower) half as well.
Fill	false	Fill each frequency slice (MIMO only; warning issued for SISO).
FaceAlpha	0.3	Patch transparency.
ShowBoundary	true	Overlay a wire boundary when Fill=true.
BoundaryColor	same as fill	Wire color when filled.
FillTightness	0	0 = convex hull per slice; 1 = exact polyshape boundary.
gminus	{}	Lower boundary cell array (SISO only).

5.5 srg_plot_3d_surface

Renders the SRG as a continuous smooth **surf** across frequency. Each slice is resampled to a common number of arc-length-parameterized points; adjacent slices are rotationally aligned before stitching.

```
1 srg_plot_3d_surface(rawdata, gplus, fmin, fmax)
2 srg_plot_3d_surface(..., 'Color', 2, 'FaceAlpha', 0.6, ...
3   'EdgeAlpha', 0.1, 'Mirror', true, 'NumPoints', 100, ...
4   'Lighting', true, 'MinDiameter', -1)
```

Option	Default	Description
Color	1	Color index or RGB triplet.
FaceAlpha	0.6	Surface transparency.
EdgeAlpha	0.1	Edge transparency.
Mirror	true	Also render the conjugate surface.
NumPoints	100	Resampling points per slice.
Lighting	true	Enable Gouraud lighting.
MinDiameter	-1	Skip slices smaller than this (auto-detected when -1).

5.6 srg_plot_3d_compare

Overlays two SRG surfaces in 3D and highlights frequency slices where the two SRGs intersect, using polyshape intersection.

```

1 srg_plot_3d_compare(rawdata1, gplus1, rawdata2, gplus2, fmin, fmax)
2 srg_plot_3d_compare(..., 'Color1', 1, 'Color2', 5, ...
3   'FaceAlpha', 0.45, 'EdgeAlpha', 0.5, 'Lighting', true, ...
4   'IntersectColor', [], 'IntersectAlpha', 0.9, ...
5   'Mirror', true, 'NumPoints', 100, 'MinDiameter', -1)

```

SISO systems are automatically detected and redirected to `srg_plot_3d` (no surface rendering needed).

5.7 srg_plot_3d_beltrami

3D wire plot in the Beltrami-Klein disk across frequencies.

```

1 srg_plot_3d_beltrami(rawdata, W, fmin, fmax, color)
2 srg_plot_3d_beltrami(..., 'Theme', 'dark')

```

Handles three cases automatically: constant matrix (replicate single curve at every frequency level), SISO (trajectory of scalar BK image), MIMO (per-frequency curve stack).

5.8 srg_plot_3d_beltrami_compare

Overlays two SRG surfaces in the BK disk in 3D, with an optional unit-disk cylinder boundary and intersection highlighting.

```

1 srg_plot_3d_beltrami_compare(rawdata1, W1, rawdata2, W2, fmin, fmax)
2 srg_plot_3d_beltrami_compare(..., ...
3   'Color1', 4, 'Color2', 5, 'FaceAlpha', 0.8, ...
4   'Lighting', true, 'IntersectColor', 1, 'IntersectAlpha', 1, ...
5   'ShowDisk', true, 'DiskAlpha', 0.06, 'DiskColor', [], ...
6   'DiskNTheta', 180, 'NumPoints', 100, 'MinDiameter', -1)

```

Option	Default	Description
ShowDisk	true	Render the unit-circle as a translucent cylinder.
DiskAlpha	0.06	Disk surface transparency.
DiskColor	gray	Disk color (RGB or index).
DiskNTheta	180	Angular resolution of the cylinder.

5.9 srg_plot_witness_3d

Overlays the minimum-distance witness segment(s) on an existing 3D SRG plot. Call after `srg_plot_3d_compare` on the same axes, using the closest-pair endpoints returned by `srg_stability_margin`.

```
1 srg_plot_witness_3d(x1, x2, freq, stab_margin)
2 srg_plot_witness_3d(..., 'Stride', 10, 'Mirror', true, ...
3   'Color', 8, 'LineWidth', 2, 'MarkerSize', 10)
```

By default only the single critical segment (at the frequency of minimum margin) is drawn, with both endpoints marked. The `Stride` option additionally draws every `Stride`-th background segment in thin grey; rendering every segment would produce a misleading surface-like artifact.

Option	Default	Description
Stride	0	Draw every k -th background segment (0 = critical only).
Mirror	true	Also draw conjugate lower-half segments.
Color	8	Segment color (palette index or RGB).
LineWidth	2	Witness segment line width.
MarkerSize	10	Endpoint marker size.

6 Style System and Figure Export

6.1 srg_style

Returns a style struct `s` consumed by all plot functions.

```
1 s = srg_style() % default theme
2 s = srg_style('Theme', 'dark')
3 s = srg_style('Theme', 'publication')
```

Available Themes

Theme	Description
default	White background, Helvetica font, minor grid.
light	Identical to default (alias for clarity).
dark	Dark background ([0.15 0.15 0.18]), pale CB colors, white grid lines.
publication	White background, Times New Roman, 14 pt, LaTeX interpreter, <code>FaceAlpha=0.35</code> .

Struct Fields

Field	Type	Description
<code>s.palette</code>	$N \times 3$	CB colorblind-safe colors used for cycling (8×3 for default/light/publication; the dark theme uses a 7×3 set of pale variants).

Field	Type	Description
<code>s.color(n)</code>	function	Returns <code>s.palette(mod(n-1, size(s.palette,1))+1, :)</code> .
<code>s.linewidth</code>	double	Default line width.
<code>s.fontsize</code>	double	Label font size.
<code>s.fontname</code>	string	Font name.
<code>s.facealpha</code>	double	Default fill transparency.
<code>s.interpreter</code>	string	'tex' or 'latex'.
<code>s.grid</code>	string	'minor' or 'on'.
<code>s.axiscolor</code>	RGB	Color for axis crosshairs.
<code>s.bgcolor</code>	RGB	Axes background color.
<code>s.cb.*</code>	RGB	Direct access to all CB color variants (<code>s.cb.red</code> , <code>s.cb.pale_blue</code> , <code>s.cb.dark_green</code> , etc.).

Colorblind-Safe Palette (Primary Colors)

Index	Name	RGB
1	CB Blue	[0.267, 0.467, 0.667]
2	CB Red	[0.933, 0.400, 0.467]
3	CB Green	[0.133, 0.533, 0.200]
4	CB Yellow	[0.800, 0.733, 0.267]
5	CB Purple	[0.667, 0.200, 0.467]
6	CB Cyan	[0.400, 0.800, 0.933]
7	CB Grey	[0.733, 0.733, 0.733]
8	CB Orange	[0.850, 0.325, 0.098]

6.2 `srg_apply_style`

Applies the style struct to the current axes.

```

1 srg_apply_style(s)
2 srg_apply_style(s, 'Crosshairs', true, 'AxisEqual', true, ...
3     'Labels', {'Re', 'Im'})

```

7 Internal Helper Functions (bg/)

These functions live in `bg/` and are called internally by the public API. They should not normally be called directly.

7.1 `beltrami_map_matrix`

Applies the BK map (6) to a square complex matrix T .

```

1 map = beltrami_map_matrix(T)

```

7.2 `beltrami_map_scalar`

Applies the scalar BK map (7), $f(\lambda) = (\bar{\lambda} - i)(\lambda - i)/(1 + \bar{\lambda}\lambda)$, element-wise to a matrix of complex values (typically eigenvalues). It is the scalar counterpart of `beltrami_map_matrix` and ships

in `bg/` for completeness; the current public pipeline maps operators via `beltrami_map_matrix` rather than this routine.

```
1 map = beltrami_map_scalar(T)
```

7.3 `beltrami_inv`

Applies the inverse BK mapping (8) to obtain the two branches g^+ and g^- of the SRG in the Gauss plane.

```
1 [gplus, gminus] = beltrami_inv(T)
```

7.4 `srg_refine_chords`

Purpose

Subdivides long chords of a Beltrami-Klein-plane boundary before it is passed to `beltrami_inv`. Called internally by `srg_compute` at every frequency slice (and for constant matrices) whenever `Refine=true` (the default); see the chord-refinement note under `srg_compute` and the discussion of the Inverse BK Mapping in Section 2.

Signature

```
1 Wrefined = srg_refine_chords(W, points, tol)
```

Argument	Type	Description
<code>W</code>	complex	Column vector of BK-plane boundary points, as returned by <code>field_of_values</code> .
<code>points</code>	int	Number of interpolated points inserted along a chord that exceeds <code>tol</code> .
<code>tol</code>	double	Maximum real- or imaginary-part step allowed between consecutive boundary points before a chord is subdivided.

Algorithm

Two passes, run consecutively: (1) walk the boundary backward and subdivide any consecutive pair whose real-part step exceeds `tol`; (2) repeat for the imaginary-part step on the result of (1). Each subdivided chord is parameterized along whichever axis (real or imaginary) has the larger spread, using direct two-point line arithmetic (not a least-squares fit, since two points determine the chord exactly), which avoids the ill-conditioning of parameterizing a near-vertical chord by its real part. If no chord in `W` exceeds `tol`, `Wrefined` is returned unchanged.

Example

```
1 H = diag([3, -2/3, 1]);
2 map1 = beltrami_map_matrix(H);
3 [W1, ~] = field_of_values(map1, 1, 32, 1); % vertex-only, normal
      matrix
4 W1r = srg_refine_chords(W1, 32, 0.05); % densified before
      beltrami_inv
5 [gp, gm] = beltrami_inv(W1r);
```

7.5 srg_export

Purpose

Exports the current figure in a publication-ready format, re-applying the `publication` theme from `srg_style` before writing. The file extension selects the format.

Signature

```
1 srg_export(filename)
2 srg_export(filename, 'Width', 8.5, 'Height', 6.5, 'DPI', 300, ...
3           'Fig', gcf, 'ApplyStyle', true, 'FontSize', [])
```

Option	Default	Description
Fig	gcf	Figure handle to export (call <code>figure(fig)</code> first).
Width	8.5 cm	Output width (8.5 cm = one IEEE column; 17.4 cm = double).
Height	6.5 cm	Output height.
DPI	300	Resolution for raster (.png) export.
ApplyStyle	true	Re-apply the publication style before export.
FontSize	[]	Override font size; [] uses the theme default.

Formats by extension

Ext	Backend
.png	<code>exportgraphics</code> at the specified DPI (crops to plot content), with a <code>getframe+imwrite</code> fallback if <code>exportgraphics</code> is unavailable (no print driver required).
.tikz	<code>matlab2tikz</code> ; <code>\input{}</code> -ready, inherits the LaTeX document font. Requires <code>matlab2tikz</code> on the path.
.pdf	Best-effort vector PDF.
.eps	Best-effort vector EPS.

Example

```
1 figure(fig);           % make the target figure current first
2 [~,~,gp,gm,~] = srg_compute(G, -2, 3, 200, 1);
3 srg_plot_gauss(G, gp, gm, 1, 200, 'Theme', 'publication');
4 srg_export('fig_srg.png', 'DPI', 300);           % always works
5 srg_export('fig_srg.tikz');                       % vector via pdflatex
6 srg_export('fig_srg.pdf', 'Width', 8.5);         % best-effort vector PDF
```

7.6 field_of_values

Computes the field of values (numerical range) boundary of a square complex matrix, from the Matrix Computation Toolbox by N. J. Higham [8]. Based on the algorithm of A. Ruhe via C. R. Johnson (1978).

```
1 [f, e] = field_of_values(B, nk, thmax, noplot)
```

Argument	Default	Description
<code>nk</code>	1	Number of leading principal submatrices to use.
<code>thmax</code>	16	Number of equally spaced angles (increase for smoother output).
<code>noplot</code>	(required)	Must be 1 (or <code>true</code>). The plotting branch has been removed from this copy (it required <code>cpltaxes</code> from the Matrix Computation Toolbox, which is not bundled). Passing 0 or omitting this argument raises an error.

7.7 `srg_to_polyshape`

Converts a complex SRG boundary vector to a `polyshape` object, preserving the original traversal order (no resorting).

```
1 [ps, x_clean, y_clean] = srg_to_polyshape(srg)
```

8 Example Scripts

8.1 Example 1 — Frequency-wise SRG (`example_01_freqwise_srg.m`)

Demonstrates the basic `srg_compute` → `srg_plot_*` → `srg_stability_margin` pipeline for three feedback pairs of increasing complexity. The MIMO loop also shows the homotopy overlay and the witness segment via `srg_plot_witness_3d`.

Loop A — SISO. Two stable systems; `points=1`. All 2D and 3D plots are demonstrated.

Loop B — MIMO 2×2 (stable). `points=500`. Includes a homotopy overlay and a 3D witness segment.

Loop C — MIMO 2×2 (unstable closed loop). `points=500`. Demonstrates SRG-based detection of instability via non-empty intersection of the two SRGs.

8.2 Example 2 — Soft SRG (`example_02_soft_srg.m`)

Demonstrates `srg_scaled_graph` for both SISO and MIMO systems, including comparison of frequency-wise slices versus the hyperbolic convex hull.

Key workflow

```
1 [W1, ~, gp1, ~, ~] = srg_compute(G1, -3, 3, 100, points);
2 [sg1_plus, sg1_minus, h1] = srg_scaled_graph(W1);
3
4 figure;
5 srg_plot_compare(sg1_plus, sg2_plus, 'Fill', true, ...
6     'Names', ["G1 hull", "G2 hull"]);
```

8.3 Example 3 — Hard SRG (`example_03_hard_srg.m`)

Demonstrates `srg_hard` for:

Case A — MIMO 2×2. G_1 has an unstable pole at $s = +1$.

Case B — MIMO 3×3. G_2 has an unstable pole and an integrator.

Key workflow

```

1 [srg1, ~] = srg_hard(G1, alphas, phis, freqs, 'InfVal', 50);
2 [srg2, ~] = srg_hard(-inv(G2), alphas, phis, freqs, 'InfVal', 50);
3
4 figure;
5 fill(real(srg1), imag(srg1), [0.85 0.32 0.10], ...
6      'FaceAlpha', 0.35, 'EdgeColor', 'none', 'DisplayName', 'G_1');
7 hold on;
8 fill(real(srg2), imag(srg2), [0.00 0.45 0.74], ...
9      'FaceAlpha', 0.50, 'EdgeColor', 'none', 'DisplayName', '-inv(G_2)');
10 axis equal; grid minor; legend;

```

8.4 Example 4 — QSR Multiplier and Export (example_04_qsr_export.m)

Demonstrates the multiplier and export layers built on top of the core computation.

Part 1 — QSR multiplier. Fits the circular (Q, S, R) multiplier of a stable MIMO 2×2 plant with `srg_qsr_multiplier` (Plot on, MaxOrder=10, SKIter=20), reporting the selected S/R orders, the overbound factor k , and how often $r > |c|$ holds across frequencies.

Part 2 — Bounding circle. Visualizes the minimum enclosing circle of the lowest-frequency SRG slice via `srg_bounding_circle`, overlaid on the corresponding `srg_plot_gauss` slice.

Part 3 — 3-D surface and export. Renders the full 3-D SRG surface with `srg_plot_3d_surface`, overlays the per-frequency minimum enclosing circle at every slice, then exports the figure to PNG (reliable, no print driver required), TikZ (vector, requires `matlab2tikz`), and PDF (best-effort, requires a working MATLAB print driver) via `srg_export`.

Key workflow

```

1 [~, ~, gplus, gminus, rawdata] = srg_compute(G, -3, 3, 200, 2000);
2 out = srg_qsr_multiplier(gplus, rawdata, 'MaxOrder', 10, ...
3      'SKIter', 20, 'Plot', true);
4
5 [c, r] = srg_bounding_circle(gplus{1}(:), 'Plot', true);
6
7 fig3 = figure;
8 srg_plot_3d_surface(rawdata, gplus, 0, 0, 'FaceAlpha', 0.9);
9 for i = 1:estpoints
10     [c_i, r_i] = srg_bounding_circle(gplus{i}(:), 'Plot', false);
11     th = linspace(0, 2*pi, 301);
12     plot3(real(c_i)+r_i*cos(th), imag(c_i)+r_i*sin(th), ...
13           rawdata.freq(i)*ones(1,301), 'r-');
14 end
15
16 srg_export('example_04_srg.png', 'Width', 17, 'Height', 14);
17 srg_export('example_04_srg.tikz', 'Width', 8.5, 'Height', 7); % if
18 srg_export('example_04_srg.pdf', 'Width', 17, 'Height', 14); % best
    matlab2tikz
    -effort

```

9 Conventions and Data Format**9.1 Cell Array Convention**

All computation functions return cell arrays indexed by frequency:

- **SISO:** `gplus{i}` is a vector of identical complex scalars (field of values of a 1×1 matrix). The representative value is `gplus{i}(1)`.
- **MIMO:** `gplus{i}` is a complex vector tracing the boundary of the numerical range at frequency i .
- **Constant matrix:** `gplus{i}` is identical for all i ; the cell array has `estpoints` copies of the same curve.

9.2 Frequency Convention

`srg_compute` (and therefore `srg_homotopy` and the `freq` column consumed by `srg_qsr_multiplier`) uses frequencies in **Hz** (`freqresp` with the 'Hz' unit). `srg_qsr_multiplier` converts internally to rad/s ($\omega = 2\pi f$) for the rational fit. `srg_hard` uses frequencies in **rad/s** (`freqresp` with `1i*omega`).

9.3 Feedback Partner Convention

For the feedback stability criterion, the two SRGs to overlay are:

- **Plant partner:** pass `G1` directly.
- **Controller partner:** pass `-inv(G2)`.

The SRGs must be disjoint for stability.

9.4 QSR Multiplier Format

`srg_qsr_multiplier` returns `out.Pi` as a struct with `Q = -1` (scalar), `S = out.S_tf` and `R = out.R_tf` (transfer functions). These correspond to the block multiplier (10) with the convention $\Pi = \begin{bmatrix} Q & S \\ S & R \end{bmatrix}$.

9.5 Output Table Fields (`rawdata`)

Field	Type	Description
<code>freq</code>	double	Frequency vector [Hz], column.
<code>maxphase</code>	double	Max singular angle of g^- [deg].
<code>minphase</code>	double	Min singular angle of g^- [deg].
<code>norminf</code>	double	$\ g^+\ _\infty$ (max gain).
<code>norminfm</code>	double	$\ g^+\ _{-\infty}$ (min gain).

10 Dependencies and Licenses

10.1 Third-Party Code

File	Author	License
<code>field_of_values.m</code>	N. J. Higham [8]	Matrix Computation Toolbox
<code>SrgTools.jl</code>	Julius P. J. Krebbekx [9]	BSD 3-Clause
<code>matlab2tikz</code> (optional)	N. Schlömer et al.	MIT (external, not bundled)

10.2 MATLAB Requirements

- MATLAB R2020b or later (required for `arguments` blocks).
- Control System Toolbox (`freqresp`, `isstable`, `tzero`, `tf`, `ss`, `feedback`).
- `matlab2tikz` is required only for the `.tikz` branch of `srg_export`.

10 References

- [1] Eder Baron-Prada, Adolfo Anta, and Florian Dörfler. On decentralized stability conditions using scaled relative graphs. *IEEE Control Systems Letters*, 9:691–696, 2025.
- [2] Eder Baron-Prada, Adolfo Anta, Alberto Padoan, and Florian Dörfler. Stability results for MIMO LTI systems via scaled relative graphs, 2025. URL <https://arxiv.org/abs/2503.13583>. To appear in IFAC World Congress 2026.
- [3] Eder Baron-Prada, Adolfo Anta, Alberto Padoan, and Florian Dörfler. Mixed small gain and phase theorem: A new view using scaled relative graphs. In *European Control Conference (ECC)*, 2025.
- [4] Eder Baron-Prada, Adolfo Anta, and Florian Dörfler. On the impact of operating points on small-signal stability: Decentralized stability sets via scaled relative graphs, 2026. URL <https://arxiv.org/abs/2603.13922>. arXiv:2603.13922.
- [5] Eder Baron-Prada, Adolfo Anta, and Florian Dörfler. Stability analysis of power-electronics-dominated grids using scaled relative graphs. *IEEE Transactions on Power Systems*, pages 1–15, 2026. doi: 10.1109/TPWRS.2026.3674752.
- [6] Eder Baron-Prada, Julius P. J. Krebbeckx, Adolfo Anta, and Florian Dörfler. Scaled graph containment for feedback stability: Soft–hard equivalence and conic regions, 2026. URL <https://arxiv.org/abs/2604.05567>. arXiv:2604.05567.
- [7] Thomas Chaffey, Fulvio Forni, and Rodolphe Sepulchre. Scaled relative graphs for system analysis. In *Proceedings of the 60th IEEE Conference on Decision and Control (CDC)*, pages 3087–3094, 2021. doi: 10.1109/CDC45484.2021.9683133. URL <https://arxiv.org/abs/2103.13971>.
- [8] Nicholas J. Higham. The Matrix Computation Toolbox. URL <http://www.ma.man.ac.uk/~higham/mctoolbox>. <http://www.ma.man.ac.uk/~higham/mctoolbox>.
- [9] Julius P. J. Krebbeckx, Eder Baron-Prada, Roland Tóth, and Amritam Das. Computing the hard scaled relative graph of LTI systems, 2025. URL <https://arxiv.org/abs/2511.17297>. arXiv:2511.17297.
- [10] Richard Pates. The scaled relative graph of a linear operator. *arXiv preprint*, 2021. URL <https://arxiv.org/abs/2106.05650>. arXiv:2106.05650.
- [11] Ernest K. Ryu, Robert Hannah, and Wotao Yin. Scaled relative graph: Nonexpansive operators via 2D Euclidean geometry, 2019. URL <https://arxiv.org/abs/1902.09788>. arXiv:1902.09788.